

Flutter introduces a neat feature called ‘hot reloading’, which uses Dart’s JIT capabilities to test new changes to the app instantly while preserving the state at the same time! Simply put – when you find a bug while testing your app, you can immediately edit the code and on the press of a button, the change is live, instantly! You don’t have to recompile this yourself or start from a different state. It’s in the same place, with your new changes. How cool is that?

Now coming back to Flutter

Flutter was initially released in May 2017, making it a very young project. It is used to develop applications for Android, iOS and Google Fuchsia. Flutter helps in building better looking UIs rapidly and easily. To understand how it works, let us take an example. We’ll talk about a real application which we deployed within just two days, for both iOS and Android.

If you haven’t downloaded and installed the SDK yet, head to <https://flutter.io/> and install Flutter on your machine. The Flutter docs are great resources to figure out how to get it installed and running on your machine.

When you are done installing, let’s get down to brass tacks. We start off by opening the console.

First, create a new project:

`flutter create`

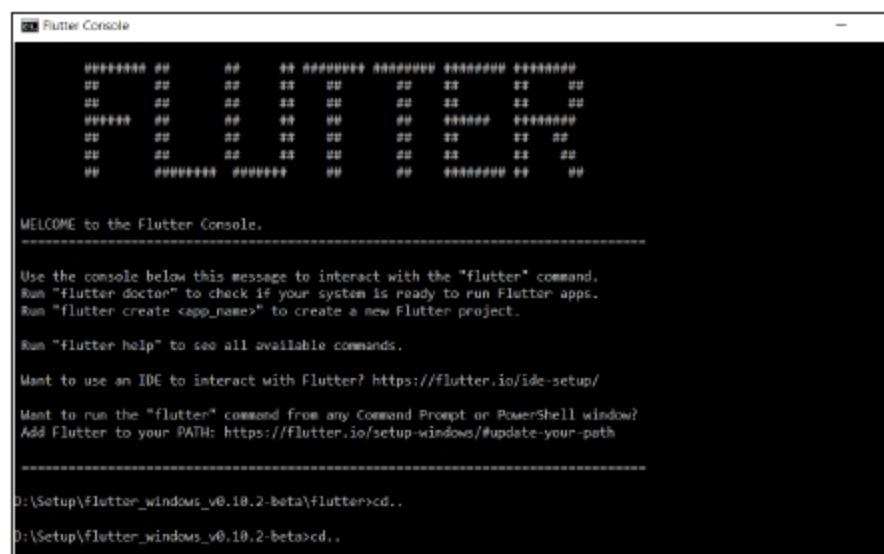


Figure 1: Flutter’s welcome screen

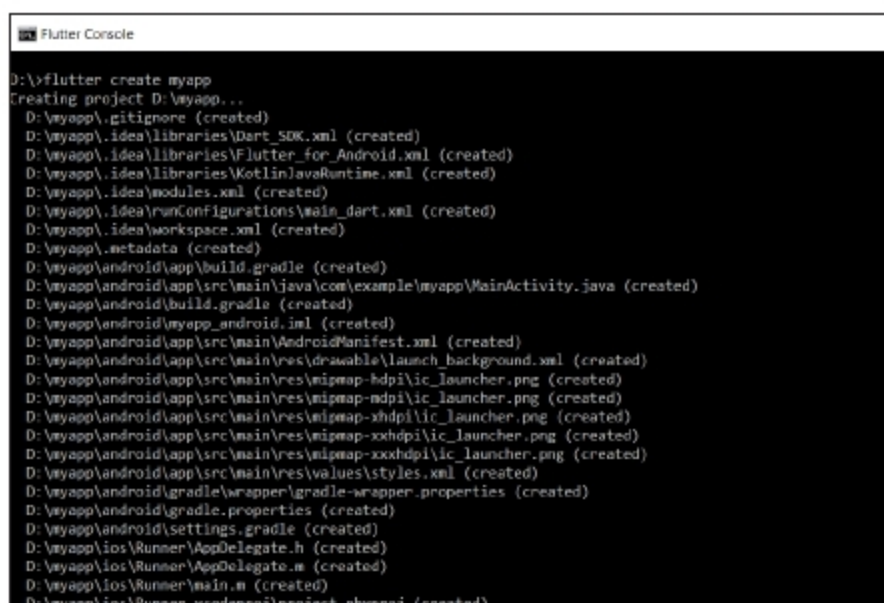


Figure 2: Creating a new app

The above command creates an application folder in the specified directory. Next, we run the app as follows:

`flutter run`

Now we can move to the application in the file directory – *lib/main.dart*, where the Dart code is, and begin editing it there.

```
import 'package:flutter/material.dart';
import 'Home.dart';

void main(){
  runApp(MaterialApp(
    title: 'LockSmith',
    home: Home(),
  ));
}
```

In here, we import the required package for Flutter – *flutter/material.dart*. Also, *Home.dart*, which contains the code to the app’s *HomePage*, directs you to *Home()* in *Home.dart*. So we start off with the *Home* class, our default view:

```
class Home extends StatelessWidget {
  // This widget is the root of your application.
  @override
  Widget build(BuildContext context)
```

This is a *StatelessWidget* that we will make use of when we don’t need to manage any form of internal state – for example, in the case of the icon, text and raised buttons.

Now we add some more components in a container widget:

```
Widget titleSection = Container(
  padding: const EdgeInsets.all(32.0),
  child: Row(
    children: [
      Expanded(
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            Container(
              padding: const EdgeInsets.only(bottom: 8.0),
              child: Text(
                'Welcome to LockSmith App',
                style: TextStyle(
                  fontWeight: FontWeight.bold,
                ),
              ),
            ),
            Text(
              'Bringing your doorbell to your phone',
              style: TextStyle(
                color: Colors.grey[500],
              ),
            ),
          ],
        ),
      ),
    ],
  ),
);
```